

Track A · Streamlit: build the *Watchlist Explorer*

Session 5 · BDS M1 · Era 2 (mid-code) · ~20 minutes · work in pairs

You will build one small app in six steps. Each step adds **one** of the five habits from the slide "*One script, five habits*". The dataset ships with Plotly — nothing to download.

The idea: six tech stocks, weekly, 2018–2019. Pick the tickers you care about, pick a window, rebase them to a common start so they can actually be compared, pin the views worth remembering, and hand the filtered data to whoever asked.

Keep `streamlit run app.py` running the whole time. Save the file → click **Rerun** (or press `R`) in the browser → see the change. That loop is the exercise.

Step 0 · Setup (2 min)

```
# macOS / Linux
python -m venv .venv && source .venv/bin/activate

# Windows (PowerShell)
python -m venv .venv; .venv\Scripts\Activate.ps1

pip install streamlit pandas plotly
```

Create an empty `app.py` in the same folder, then:

```
streamlit run app.py
```

Your browser opens on `localhost:8501`. It is blank — that is correct.

Step 1 · Load the data once, not on every click

Streamlit reruns your **whole script**, top to bottom, every single time you touch a widget. Without a cache, that means re-loading the data on every click.

```

import time

import pandas as pd
import plotly.express as px
import streamlit as st

st.set_page_config(page_title="Watchlist Explorer", layout="wide")
st.title("Watchlist Explorer")
st.caption("Six tech stocks, weekly, 2018-2019. Indexed to 1.00 on 2018-01-01.")

@st.cache_data
def load_data():
    time.sleep(2) # stand-in for a slow API / database call – delete in a real app
    wide = px.data.stocks()
    wide["date"] = pd.to_datetime(wide["date"])
    long = wide.melt(id_vars="date", var_name="ticker", value_name="price")
    return long.sort_values(["ticker", "date"])

df = load_data()
st.dataframe(df.head())

```

Try it: reload the page. The first run takes 2 seconds; every run after that is instant — same function, no second call. Now delete `@st.cache_data`, save, and click around: 2 seconds, *every time*. Put it back.

The point: `@st.cache_data` keys the cache on the function's arguments. Same arguments → stored result. That one line is what makes a rerun-everything model usable.

Step 2 · Three widgets in the sidebar

Replace `st.dataframe(df.head())` with:

```

with st.sidebar:
    st.header("Controls")
    tickers = st.multiselect(
        "Tickers",
        options=sorted(df["ticker"].unique()),
        default=["AAPL", "MSFT", "AMZN"],
    )
    start, end = st.slider(
        "Date range",
        min_value=df["date"].min().date(),
        max_value=df["date"].max().date(),
        value=(df["date"].min().date(), df["date"].max().date()),
    )
    rebase = st.checkbox("Rebase to 100 at window start", value=True)

view = df[df["ticker"].isin(tickers) & df["date"].dt.date.between(start, end)].copy()

if view.empty:
    st.info("Pick at least one ticker in the sidebar.")
    st.stop()

if rebase:
    view["price"] = view.groupby("ticker")["price"].transform(lambda s: s / s.iloc[0] *
100)

st.dataframe(view.head())

```

The point: a widget is just a function that *returns a value*. `tickers` is a list, `start / end` are dates, `rebase` is a bool — plain Python from there on. No callbacks, no event handlers.

`st.stop()` is the guard rail: with nothing selected, the code below would crash on an empty frame.

Step 3 · A chart that reacts to every widget

Replace that last `st.dataframe(view.head())` with:

```

perf = view.groupby("ticker")["price"].agg(["first", "last"])
perf["return_%"] = (perf["last"] / perf["first"] - 1) * 100
best = perf["return_%"].idxmax()

c1, c2, c3 = st.columns(3)
c1.metric("Tickers", len(tickers))
c2.metric("Weeks in window", view["date"].nunique())
c3.metric(f"Best: {best}", f"{perf.loc[best, 'return_%']:.1f}%")

fig = px.line(
    view,
    x="date",
    y="price",
    color="ticker",
    labels={"price": "Rebased (start = 100)" if rebase else "Index (2018-01-01 = 1.00)",
           "date": ""},
)
fig.update_layout(height=420, margin=dict(t=10, b=0), legend_title_text="")
st.plotly_chart(fig)

```

Try it: set the window to 2019 only, with *Rebase* on. The "best" ticker changes — over the full two years it is MSFT (+78.8%), but over 2019 alone it is AAPL (+92.8%).

The point: this is last session's lesson made interactive. Rebasing is choosing a benchmark, and the checkbox hands that choice to the audience instead of freezing it in a PNG. Nothing in this block knows a widget exists — it just reads `view`, and the rerun does the rest.

Step 4 · Pin a view so it survives the rerun

Rerun-everything has a cost: normal Python variables are wiped on every interaction. `st.session_state` is the dictionary that isn't.

```

if "pinned" not in st.session_state:
    st.session_state.pinned = []

left, right, _ = st.columns([1, 1, 4])
if left.button("Pin this view"):
    st.session_state.pinned.append(
        {
            "tickers": ", ".join(tickers),
            "from": start,
            "to": end,
            "best": best,
            "return_%": round(float(perf.loc[best, "return_%"]), 1),
        }
    )
if right.button("Clear pins"):
    st.session_state.pinned = []

if st.session_state.pinned:
    st.subheader("Pinned views")
    st.dataframe(pd.DataFrame(st.session_state.pinned), hide_index=True)

```

Try it: pin 2018, then pin 2019, and compare the two rows.

The point: `if "pinned" not in st.session_state` runs on every rerun — that is why the initialisation has to be guarded. A button returns `True` only on the rerun it triggered, so `if st.button(...)` is how you say "do this once, now".

Step 5 · Let them take the data home

```

st.download_button(
    "Download filtered data (CSV)",
    data=view.to_csv(index=False).encode("utf-8"),
    file_name="watchlist.csv",
    mime="text/csv",
)

```

The point: two lines turn an explorable into something a colleague can open in Excel. It is also the honest move — the audience can check your filtering instead of taking it on trust.

Done. Five habits, roughly 70 lines of Python, zero HTML.

Step 6 · Ship it (optional, 10 min)

6a · Write the shopping list

```
pip freeze > requirements.txt
```

Open the file. It should be short. If it runs to hundreds of lines you were not inside the virtual environment when you installed — delete it and type the three names by hand instead:

```
streamlit  
pandas  
plotly
```

Your project folder now needs to contain exactly two files:

```
watchlist-explorer/  
├─ app.py  
└─ requirements.txt
```

6b · Push it to GitHub

GitHub is the hand-off point: Streamlit Cloud does not take files from your laptop, it reads them from a repository. Pick whichever route you prefer — the result is identical.

Route 1 — the browser (no git, no terminal)

1. Sign in at **github.com** (create a free account if you need one).
2. Top right + → **New repository**. Name it `watchlist-explorer`, leave it **Public**, add nothing else, click **Create repository**.
3. On the empty repo page, click the **uploading an existing file** link.
4. Drag `app.py` and `requirements.txt` into the box — those two files only. Do **not** drag the `.venv` folder; it is large, machine-specific, and the whole point of `requirements.txt` is that nobody needs it.
5. Click **Commit changes**.

Route 2 — the terminal (if you have git)

Create the empty repository on github.com first (steps 1-2 above), then, from your project folder:

```
git init  
echo ".venv/" > .gitignore  
git add app.py requirements.txt .gitignore  
git commit -m "Watchlist Explorer"  
git branch -M main  
git remote add origin https://github.com/<your-username>/watchlist-explorer.git  
git push -u origin main
```

If the push asks for a password, it does not want your account password — GitHub stopped accepting those for git in 2021. It wants a **personal access token** (*Settings → Developer settings → Personal access tokens*). The painless alternative is to run `gh auth login`, or install GitHub Desktop, and let either handle the sign-in for you.

Never commit a key. Today's app has none. The moment yours calls an API, the key belongs in `.streamlit/secrets.toml`, and that file belongs in `.gitignore`. A key pushed once stays in the repository's history even after you delete the line.

6c · Deploy

Sign in at share.streamlit.io with GitHub → **Create app** → "Yup, I have an app" → choose your repository, the `main` branch, and `app.py` → **Deploy**. Live at `https://[yourname].streamlit.app` in a few minutes.

To update it later, push a new commit — the deployed app rebuilds itself. If the deploy fails, read the log on the right: it is almost always a package missing from `requirements.txt`.

Step 7 · Real data, behind a key · 15 min

Everything so far used a dataset that ships with Plotly. Real data usually sits behind a key — and a key changes how you organise the project, because one file must never leave your machine.

7a · Get a key · 1 min

Go to alphavantage.co/support/#api-key, enter an email and one line about your project. The key appears on the page immediately — no credit card, no confirmation email.

Free tier: 25 requests a day, 5 a minute. That limit is not a nuisance, it is the reason Step 1's cache exists. One request per ticker, so stick to two or three.

7b · The folder now has four things

```
watchlist-explorer/
├── .streamlit/
│   └── secrets.toml      <-- ONLY on your machine, never on GitHub
├── .gitignore
├── requirements.txt
└── app.py
```

`requirements.txt` — one new line:

```
streamlit
pandas
plotly
requests
```

`.gitignore` — this file is what keeps you out of trouble:

```
# never commit secrets
.streamlit/secrets.toml

# python noise
__pycache__/
*.pyc
.venv/
```

`.streamlit/secrets.toml` — your real key, on your machine only:

```
ALPHAVANTAGE_API_KEY = "paste-your-real-key-here"
```

7c · Read the key, fetch real prices

Replace Step 1's `load_data()` with this. Everything below it stays exactly as it is.

```

import requests

@st.cache_data(ttl=3600)
def load_prices(symbol):
    r = requests.get(
        "https://www.alphavantage.co/query",
        params={
            "function": "TIME_SERIES_DAILY",
            "symbol": symbol,
            "apikey": st.secrets["ALPHAVANTAGE_API_KEY"],
            "outputsize": "compact",
        },
        timeout=10,
    )
    payload = r.json()

    series = payload.get("Time Series (Daily)")
    if series is None:
        # the API answers 200 even when it refuses
        raise RuntimeError(payload.get("Information") or payload.get("Note") or
"Unexpected response")

    out = (pd.DataFrame(series).T
          .rename(columns={"4. close": "price"})[["price"]]
          .astype(float)
          .rename_axis("date")
          .reset_index())
    out["date"] = pd.to_datetime(out["date"])
    out["ticker"] = symbol
    return out.sort_values("date")

@st.cache_data(ttl=3600)
def load_data(symbols=("AAPL", "MSFT", "IBM")):
    return pd.concat([load_prices(s) for s in symbols], ignore_index=True)

```

Three things worth stopping on:

- `st.secrets["..."]` reads the local file now and the Cloud settings after you deploy. Same line, both places. You never write a key into `app.py`.
- `ttl=3600` expires the cache after an hour. Without it you would burn through 25 requests in one afternoon of clicking.
- The API answers **200 OK** even when it refuses you. Over the daily limit, it returns a normal response with an `Information` message and no data. Checking the status code would tell you everything is fine. Check the *payload*.

7d · Prove the secret is safe — before you push

```
git init
git add .
git status
```

Read the list. `secrets.toml` must not appear. If it does, `.gitignore` is missing or misspelled — fix it before going further, because a key pushed once stays in the history forever.

```
git commit -m "Watchlist Explorer with live data"
git branch -M main
git remote add origin https://github.com/<your-username>/watchlist-explorer.git
git push -u origin main
```

7e · Give the key to Streamlit Cloud

Your app is now on GitHub *without* its key, so the deployed version has nothing to read. Deploy as in Step 6, then:

Advanced settings → **Secrets** → paste the same line:

```
ALPHAVANTAGE_API_KEY = "paste-your-real-key-here"
```

Streamlit Cloud stores it outside the repository and hands it to `st.secrets` at runtime.

The point: the key exists in exactly two places — your laptop and the hosting platform's settings — and in neither of them is it part of your code. That separation is the whole idea, and it is the same idea whether the secret is an API key, a database password, or a customer list.

If your key does not work yet

Use `apikey=demo` instead. It works for `IBM` only, which is enough to see the app running while you sort the key out.

Make it yours (pick one, 5 min)

EXTENSION	HINT
Add a moving average	<code>view.groupby("ticker")["price"].transform(lambda s: s.rolling(4).mean())</code> — plot it as a second trace
Show the best/worst week per ticker	<code>view.groupby("ticker")["price"].pct_change()</code> → <code>st.dataframe</code>
Two windows side by side	<code>st.columns(2)</code> , one chart in each
Swap the dataset entirely	<code>px.data.gapminder()</code> , <code>px.data.tips()</code> , or <code>pd.read_csv(<your own URL>)</code> — the five habits do not change
Cache a <i>computation</i>	Put <code>@st.cache_data</code> on a slow function that takes <code>tickers</code> as an argument, and watch the cache key change

When it breaks

SYMPTOM	CAUSE
App is slow on every click	<code>@st.cache_data</code> missing, or the loader takes an unhashable argument
<code>IndexError</code> on <code>s.iloc[0]</code>	The filter returned an empty frame — the <code>st.stop()</code> guard is missing or placed after the code that crashes
Pinned list empties itself	<code>st.session_state.pinned = []</code> written without the <code>if "pinned" not in ...</code> guard
Widget "does nothing"	You read its value <i>above</i> where the widget is defined — the script runs top to bottom
Blank page, no error	Check the terminal, not the browser: tracebacks print there

Then compare with Track B

You built this in Python, one habit at a time, and you can point at the line that does each thing. Now describe the *same* app to Google AI Studio in one paragraph and see what comes back.

Three questions to answer as a pair:

1. Which was faster to a first working version — and by how much, honestly?
2. Which one could you **debug** at 9am tomorrow when a stakeholder says the numbers look wrong?
3. Which one would you put your name on?

Appendix · the complete `app.py`

```
"""
Watchlist Explorer – Session 5, Track A (Era 2 · Streamlit)
BDS M1 · Hamid Bekamiri

Run it with:  streamlit run app.py
"""

import time

import pandas as pd
import plotly.express as px
import streamlit as st

st.set_page_config(page_title="Watchlist Explorer", layout="wide")
st.title("Watchlist Explorer")
st.caption("Six tech stocks, weekly, 2018-2019. Indexed to 1.00 on 2018-01-01.")

# --- 1. cache the expensive load -----
@st.cache_data
def load_data():
    time.sleep(2) # stand-in for a slow API / database call – delete in a real app
    wide = px.data.stocks()
    wide["date"] = pd.to_datetime(wide["date"])
    long = wide.melt(id_vars="date", var_name="ticker", value_name="price")
    return long.sort_values(["ticker", "date"])

df = load_data()

# --- 2. sidebar: multiselect + slider + checkbox -----
with st.sidebar:
    st.header("Controls")
    tickers = st.multiselect(
        "Tickers",
        options=sorted(df["ticker"].unique()),
        default=["AAPL", "MSFT", "AMZN"],
    )
    start, end = st.slider(
        "Date range",
        min_value=df["date"].min().date(),
        max_value=df["date"].max().date(),
        value=(df["date"].min().date(), df["date"].max().date()),
    )
```

```

rebase = st.checkbox("Rebase to 100 at window start", value=True)

view = df[df["ticker"].isin(tickers) & df["date"].dt.date.between(start, end)].copy()

if view.empty:
    st.info("Pick at least one ticker in the sidebar.")
    st.stop()

if rebase:
    view["price"] = view.groupby("ticker")["price"].transform(lambda s: s / s.iloc[0] *
100)

# --- 3. headline numbers + a chart that reacts to every widget -----
perf = view.groupby("ticker")["price"].agg(["first", "last"])
perf["return_%"] = (perf["last"] / perf["first"] - 1) * 100
best = perf["return_%"].idxmax()

c1, c2, c3 = st.columns(3)
c1.metric("Tickers", len(tickers))
c2.metric("Weeks in window", view["date"].nunique())
c3.metric(f"Best: {best}", f"{perf.loc[best, 'return_%']:+.1f}%")

fig = px.line(
    view,
    x="date",
    y="price",
    color="ticker",
    labels={"price": "Rebased (start = 100)" if rebase else "Index (2018-01-01 = 1.00)",
           "date": ""},
)
fig.update_layout(height=420, margin=dict(t=10, b=0), legend_title_text="")
st.plotly_chart(fig)

# --- 4. session_state: pinned views survive reruns -----
if "pinned" not in st.session_state:
    st.session_state.pinned = []

left, right, _ = st.columns([1, 1, 4])
if left.button("Pin this view"):
    st.session_state.pinned.append(
        {
            "tickers": ", ".join(tickers),
            "from": start,
            "to": end,
            "best": best,
            "return_%": round(float(perf.loc[best, "return_%"]), 1),

```

```

    }
)
if right.button("Clear pins"):
    st.session_state.pinned = []

if st.session_state.pinned:
    st.subheader("Pinned views")
    st.dataframe(pd.DataFrame(st.session_state.pinned), hide_index=True)

# --- 5. let the stakeholder take the data home -----
st.download_button(
    "Download filtered data (CSV)",
    data=view.to_csv(index=False).encode("utf-8"),
    file_name="watchlist.csv",
    mime="text/csv",
)

```